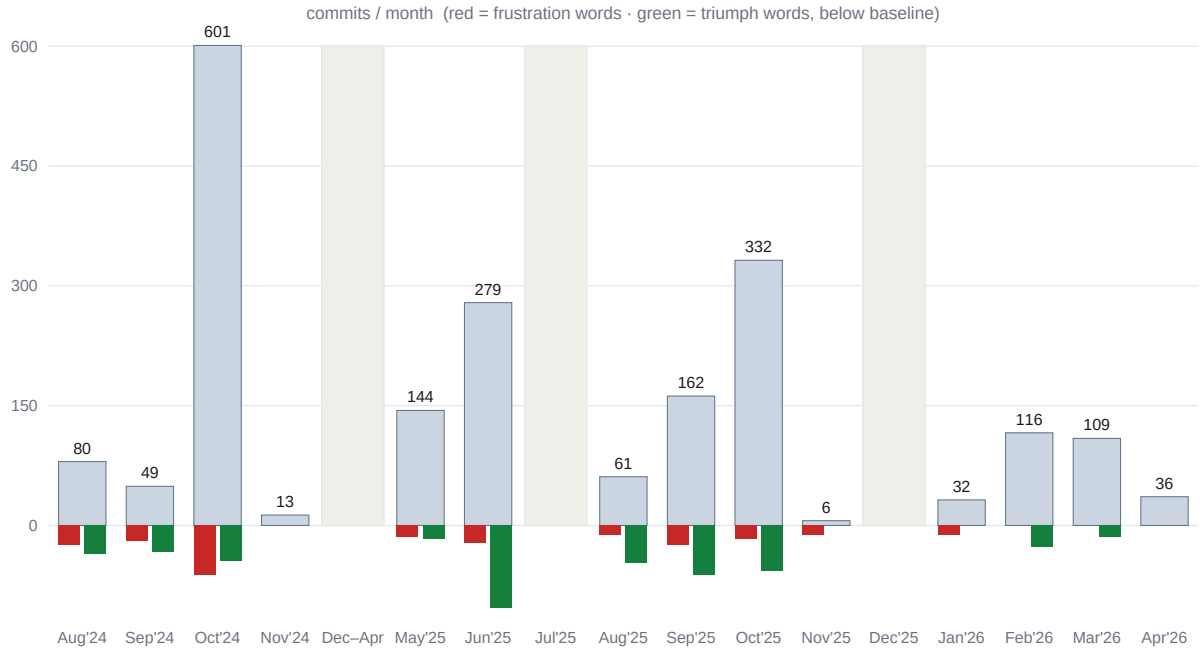
The MonsterBox Story

Two years of building alongside AI — told through 2,020 commits, from a hand-typed fff at midnight to an AI that obeys a constitution before it can commit.

2,020	601	13 → 70	11+
commits, 20 months	in Oct 2024 alone	avg msg length (chars/mo)	dev tools came & went



The project's heartbeat and mood — commit volume per month, with frustration (red) and triumph (green) word-counts beneath each bar. This whole document is reconstructed from the git history; nothing is invented.

First, what is MonsterBox?

It's Halloween night in a garage in Coralville, Iowa. A life-sized vampire — **Orlok** — turns his head to track a kid coming up the driveway. His eyes glow. As the kid gets close, Orlok leans in, jaw moving in time, and *speaks*: not a recorded clip, but a live, AI-generated reply to whatever the kid just said. A video flickers across a tombstone, lights pulse with the audio, a second creature stirs. Nobody is backstage pulling levers. It's all **MonsterBox**.

MonsterBox is an **animatronic control platform** — the software brain that drives physical Halloween creatures. It runs on a Raspberry Pi 4B wired to servos, motors, linear actuators, LEDs, sensors, a camera, speakers, and a microphone. From a web dashboard you build *characters* (Orlok the vampire, Mina, Sir Dragomir, the Groundbreaker, PumpkinHead), define their hardware *parts*, choreograph *scenes*, and then turn them loose to perform, react, and hold a conversation on their own.

What you can actually do with it on Halloween today:

- **Choreograph performances in the Animation Studio** — a timeline editor where you drag servo moves, motor runs, lights, audio, and pauses into a scene, then hit play. Scenes can loop all night or fire on a sensor trip. - **Make a creature talk and listen** — built-in **text-to-speech** gives each character a voice; **speech-to-text** lets it *hear* a guest. Drop a `sayThis` step into a scene for scripted lines. - **Hold a real conversation** — an `askAI` step sends what the guest said to a large language model and speaks the reply back, so the monster improvises. The jaw animates to the speech automatically. - **Track and follow people** — a USB camera with OpenCV motion detection drives **head-tracking**, so a character's gaze follows movement in the room. - **Project video and play media** — the **Goblin** subsystem drives synchronized video/playlist playback on external displays (a face on a screen, a scene on a wall). - **Light and move everything** — LEDs, PWM servos (via PCA9685), motors, and linear actuators, all controllable live from the dashboard or sequenced in a scene. Toggle features like jaw-sync, head-tracking, and "parrot" mode from a single panel.

In short: **video, lights, motion, voice, hearing, and AI conversation — all coordinated from one web app on a $75 computer.**

The cast on Halloween night

MonsterBox doesn't run one creature. It runs an ensemble, and they share a world — a gothic tableau staged at **Warner Castle** (the family's own house). The dialogue is real; every quote below is lifted verbatim from the characters' scene files.

- **Orlok** — Count Orlok of the Carpathians, "Ruler of Walechia," a Nosferatu-styled vampire lord and the most hardware-rich character (12 parts: dual linear-actuator arms, an elbow and a forearm servo, a swiveling head, the glowing "Hand of Azura," the IR-night-vision "Eye of Orlok"). He greets guests with *"Welcome to Warner Castle, mortal! I am Count Orlok, and tonight your soul belongs to me!"* — and when the mood turns to old grudges, *"I am Orlok of the Carpathians — Ruler of Walechia. Feel my wrath, Turk!"* and *"Release the wolves! Go my Pets, do your Worst!"*

- **Mina** — a coffin-bound vampire who rises through a motorized door, eyes lit by a laser, a "Burning Rose" beside her, speaking in a voice literally named *The Siren's Voicemail*: *"I have risen from eternal slumber. The darkness calls, and I answer!"* Her story is bound to Orlok's — guests can ask her *"Who is Orlok and what has he done to you?"* and *"What ancient curse binds you to this coffin?"*, to which she answers, *"My story is long and filled with sorrow."* She is the Mina to his Count: the horror and the heartbreak at the center of the scene.
- **Sir Dragomir** — the knight, wearing the face of John Hunyadi (the real Hungarian-Wallachian general who fought the Ottomans). He's the historical counterweight to the vampire warlord — the Turk-fighter Orlok taunts across the yard.
- **Groundbreaker** — a creature that claws up out of the ground and talks trash: *"You want some of this? I'll tear you apart and bury you in pieces!"* (the scene is, in fact, named "Groundbreaker Insult Loop").
- **PumpkinHead** — the oldest, simplest archetype, walking terror itself: *"Fear me! I am the terror that walks in darkness!"*

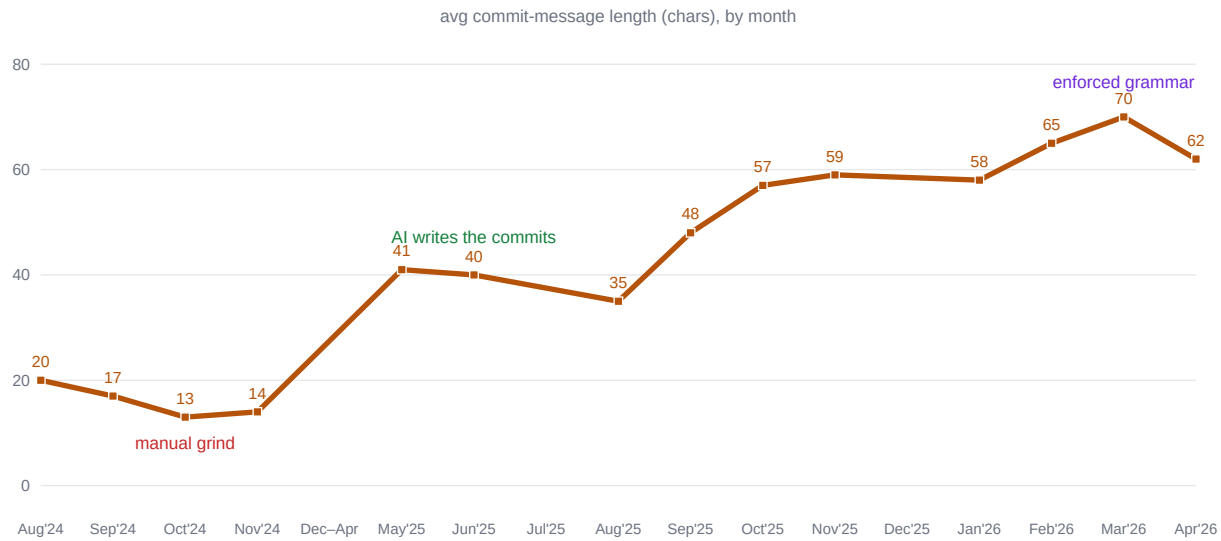
Underneath the horror there's a family running the show: parrot mode pipes in personalized Easter eggs — *"Hello Polly!! Welcome to Warner Castle!!!"*, *"Kiley is the Koolest and Nugget is Queen Skribidi!"* The whole point of the platform is that all of these — different hardware, different voices, different personalities — run on the *same* engine, react to the *same* guests, and perform on the *same* night. That ensemble is exactly why "character independence" stopped being a nicety and became the architecture: when five creatures share one codebase, one of them hardcoded as the favorite breaks the other four.

It didn't start anywhere near here

The first version, in August 2024, was the opposite of all that: one animatronic (a demon named **Baphomet**), a couple hundred lines of hand-written Node.js, three flat JSON files, and commit messages like `fff` and `embracesuck` typed at midnight before the holiday. No AI conversation, no video subsystem, no head-tracking, no multi-character support, no tests — one person copy-pasting code, wiring motors, and hoping it held for one night.

Getting from there to here took two years, and the road runs exactly parallel to how AI coding tools evolved in real time. So this isn't really a history of an animatronic platform. It's a case study in *what it was actually like to build software with AI between 2024 and 2026*, reconstructed from all 2,020 commits — as the tools went from a clever autocomplete that broke the lights, to a swarm of agents that over-built everything overnight, to a governed collaborator that finally earned trust. The lessons are collected at the end. The short version: **the AI got dramatically more capable, and the human's job shifted from writing the code to governing the thing that writes it.**

How to read this history



AI Signature — average commit-message length

You don't need to read the code to see the story — it's written into the commit messages themselves. The single most telling number in the whole repository is the **average length of a commit subject line**, measured month by month:

Period	Avg. message	Representative commit
Oct 2024	**13 chars**	`fff` · `n` · `scnese` · `embracesuck`
May 2025	**41 chars**	`Add sound controller with Python process management and audio playback`
Oct 2025	**57 chars**	`feat: integrate Copilot-driven testing with MCP tools`
Mar 2026	**70 chars**	`v7.1.0: [calibration] fix servo invert to mirror within calibrated bounds`

The messages get longer, more structured, and more descriptive — not because you started writing more, but because *something else started writing them for you*, and then because you built a system that **required** them to be good. That arc — from a human hammering keys at midnight to a governed collaboration between human and machine — is the whole story. Here it is in seven acts, told four ways at once: what the **software** did, what **tools** built it, what the **commits** say, and how it all **felt**.

Prologue — Before It Was MonsterBox: The Baphomet Origin

MonsterBox didn't begin as MonsterBox. Its git history *is* the continuous lineage of one demon. The root commit (Aug 15, 2024) opens with a throwaway placeholder character — `"Scary Pete" — "Someone with buttholes for eyes"` — and a flat, single-character data layout: one `data/characters.json`, one `parts.json`, one `scenes.json` sitting at the repo root, describing exactly one bot with five parts (two arms, two legs, glowing eyes). This is the shape of a project built to run *one* animatronic.

By **day two** (Aug 16, `Motor Integrated halfway`) the placeholder is gone and the real subject appears: **Baphomet**. Two days after that it's briefly renamed **"Lord Satan Production"** — Baphomet's alter ego — before settling back. For the first stretch of its life, this repository was simply *the code that runs Baphomet.*

The "fork one into many" you remember is real, and it happened **gradually, in place** — not as a separate repo merged in, but as the original single-bot codebase being generalized outward:

> **1. One bot (Aug 2024):** Baphomet only, flat root-level data files. 2. **A small troupe (by Nov 2024):** the roster grows to **Baphomet + Coffin Breaker + PumpkinHead** — `Post Halloween Commit - Baphomet` is the last time the founding character is named in the log. 3. **The unification (May 5, 2025):** `First rev to sync all Bots to same codebase` — the deliberate pivot from per-bot code to one shared engine. 4. **The platform (2026):** per-character `data/character-{id}/` directories, schemas, and the canonical resolver — *any* character, no hardcoding.

There is a poignant footnote: **Baphomet is no longer in the roster.** Today's characters are PumpkinHead, Mina, Orlok, Sir Dragomir, and Groundbreaker. The demon that started it all was retired along the way, and **Orlok** (char_id 3) became the dominant character — so dominant that "hardcoded to Orlok" is now the single most-warned-against bug class in `CLAUDE.md`. The whole Character Independence crusade of 2026 is, in a sense, the project working to ensure no *future* character ever becomes as load-bearing as Baphomet once was. The origin animatronic left its fingerprint on the architecture precisely by being the thing everything was once wired to.

> Note: the full git graph contains three parentless root commits. Only one (Aug 15 2024) is the application's origin; the other two — Sept 30 2025 and Apr 19 2026 — are MkDocs / `gh-pages` documentation-deploy branch roots, not a separate codebase.

Act I — The Hand-Built Origin (Aug 2024)

The repository opens the way every project does — `Initial commit`, `first commit`, `Initial commit` again (three of them; already a little chaotic). The first tool fingerprint appears on day two: a `.idea/` directory (Aug 16). You were in JetBrains, hand-wiring a Raspberry Pi to servos and LEDs, and the commits read like a heartbeat monitor:

```
Fully Functional Scenes -> Broken - splitting up apps.js into multiple files -> Final Update 8.15 Working! -> Motor Integrated halfway -> Broken but expanded pre CodeAnywhere -> FullyFunctionalPID
```

The software at this stage was already ambitious for a hand-built project: a scene system (`scene-form.ejs`), motor integration, a PID control loop (`FullyFunctionalPID`, Aug 18), sound, sensors, LEDs. But it was monolithic — the recurring commit `splitting up apps.js into multiple files` shows a single giant file being pried apart by hand.

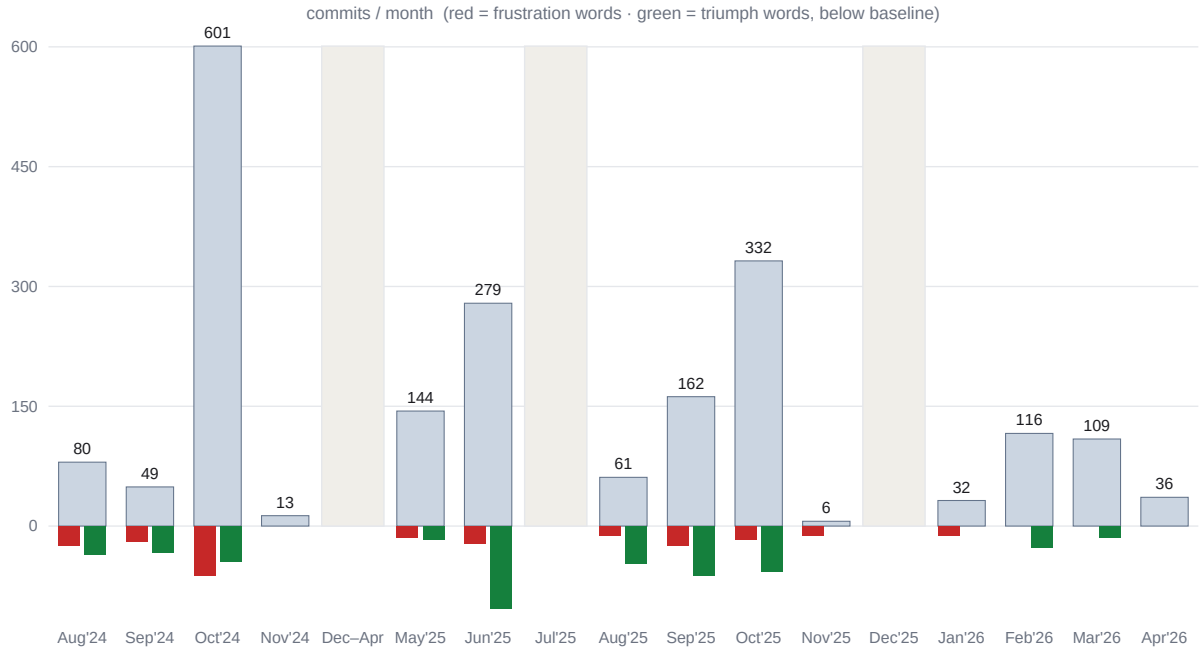
The AI relationship is captured in the most honest commit in the entire history, on Aug 18:

```
End of Claude: Broke LED and Scenes
```

That's early-AI in five words. The assistant could generate a block of code, but it had no model of your *system*, so it confidently broke the LEDs and the scene player — and you had to notice, name it, and clean up. Days later: `First ClaudeDev Update`. There are **90 all-caps status commits** (`WORKING`, `BROKEN`, `STABLE`) across the project's life and most cluster right here: without tests, every commit is a manual save-point before the next thing breaks.

Sentiment: already 6 frustration markers and 11 triumph markers in the first fortnight — a builder oscillating between *it works!* and *it's broken again.*

Act II — The October Grind (Oct 2024)



Activity & mood, by month

October 2024 is the most violent month in the project's history: **601 commits** — more than double any other month — at the *shortest* average length ever (13 chars). A `.vscode/` directory appears (Oct 5); the commit log mentions `CodeAnywhere` and `codeanyapp.com` as you tried to edit the Pi remotely. This is the run-up to your first real Halloween deadline, and you can feel the panic:

> `fff · n · scnese · logz10 · quiet errors · loop fixes · testing again · embracesuck · good time · sounds again · resolutions · update from garage`

And on Halloween itself, the rawest the log ever gets:

> `fuckucamfix · asdfuckoutside · fuckuwaziting · sounds fucked · Revert "sounds fucked" · fuckyou · Fuckyou Replica · more whatever`

22 frustration markers — the all-time monthly peak. These are commits made by a human, on real hardware, at midnight, with a deadline bearing down. No AI writes `update from garage`.

The software grew fast and messy. This is when TTS first lands (`First Draft - full TTS functionality in Character`, Oct 25), when head tracking arrives (`feat: add head tracking scrypt and route`, Oct 22 — note the typo, human-typed), and `motion tracking ready` (Oct 27). A human collaborator, `teodor`, contributes the project's first genuinely *engineered* commit message — `feat: upgrade servo control.py for smooth turn head` — a quiet preview of a discipline that wouldn't become the norm for another 18 months. There's also a massive net **deletion of ~769k lines** this month: `node_modules` and vendored assets being pulled out of tracking, the first sign of the bloat-vs-discipline tension that recurs throughout.

October ends at `resolutions`. The bots ran for Halloween. A final November burst of GPIO work (`Conversion to gpiozero`, `Post Halloween Commit - Baphomet`) — and then the project goes **dark for five months.**

Act III — The Silence & the Re-Founding (Nov 2024 -> May 2025)

No commits from late November 2024 until May 2025 (visible as the shaded gap in the timeline chart). When the project returns, it returns with a thesis. The very first commit of the new era:

> `First rev to sync all Bots to same codebase 2025 First general merge of all bots`

This is the moment MonsterBox stops being "the code that runs my one animatronic" and becomes a **platform** meant to run *any* character — the seed of what CLAUDE.md now calls **Character Independence**. The first multi-character data appears immediately: `Add initial character and part configurations for Orlok, Coffin Breaker, and PumpkinHead`.

And look what happens to the commit messages the instant the project resumes:

```
Add character form view with image preview and dual-select for parts/sounds Add
sound controller with Python process management and audio playback functionality Add
camera routes with stream, control, and head tracking functionality
```

Overnight the average message length **triples** (13 -> 41 chars). Nobody decided to type more. This is the fingerprint of AI now *authoring and narrating* whole changes — full imperative summaries of a diff it just wrote. MkDocs documentation appears (`MkDocs deployment`, `auto-deploy workflow`), another sign of AI-assisted scaffolding producing artifacts a hurried human rarely writes by hand.

Act IV — The Multi-Agent Experiment (May–Jun 2025)

This is the most fascinating chapter, because it's where you stopped using AI as a tool and started managing it as a **workforce**. Within days in early June, the repository sprouts config directories for an entire arsenal of agent tools — `.cursor/`, `.windsurf/`, `.roo/`, `.augment/`, `.taskmaster/` — almost all appearing **June 5–6, 2025** (see the cluster in the tool-timeline chart). The infrastructure lands in a rush:

- `Restructure for new AI, hardware, MCP and dev environment. Adding in Taskmaster-AI` - `Add MCP configuration with environment variable references` - `Add Augment Code remote agent implementation packages` - `🤖 Add Three Independent Augment Remote Agents - Complete Setup` - `📚 Add comprehensive documentation for Three Independent Augment Remote Agents`

You were running **three independent AI agents in parallel**, coordinated through MCP servers, against a numbered task backlog (`task-master-ai`, later migrated to Augment's built-in system). For a few weeks the commit log reads like a ticket feed generated by the agents themselves:

```
Complete Task 16: TaskMaster + MkDocs Integration Complete Task 17: Core AI
Integration (API Clients) - 100% DONE Complete Task 19 - GPIO Abstraction with I2C
Complete Task 20: AI Configuration & Management UI System - 100% DONE
```

The 🤖-and-`100% DONE` style is unmistakably machine-written — a model reporting its own completion. Across the whole history there are **227 commits carrying an AI co-author / "generated with" / 🤖 trailer**, and the bulk originate here.

The software leaps forward. This era brings the **ChatterPi conversation system** (`feat: ChatterPi Interactive Conversation System - Complete Foundation`, June 8) — real two-way voice interaction built on OpenAI GPT — plus **STT** (`working on STT`, June 14), **jaw animation** (`working basic jaw animation`, June 12), and **Playwright** end-to-end testing (`.config` appears June 20). At `v0.1.0-baseline` (June 6) the codebase is a lean **312 files**.

But it's also the most emotionally manic month. June 2025 has the *most triumph markers of all time (40)* sitting right next to some of the funniest frustration in the log:

```
MCP fixes for the billionth time · tailwind - mid conversion which sucks · fix the damn camn · fucked websockets · working basic jaw animation - animation sucks.
```

`working basic jaw animation - animation sucks.` is the whole era in one line: *it works, and I hate how it works.* And the first AI-specific discipline appears too — `Security: Replace hardcoded API keys with environment variable references in MCP config files` — because once agents commit on your behalf, leaked secrets become a live risk.

Act V — The Microservices Debacle (Aug–Sep 2025)

This is the chapter that earns the cliché *be careful what you wish for*. Late summer 2025 is the "just let the model fix it" phase. The signature commit, Sept 1, names the tool outright:

```
Major automated bug fix GPT5
```

GPT-5 had only just been released. It was, in your words, *too young* — immensely capable, with no instinct for restraint — and it was handed broad, semi-autonomous control of a hardware codebase. What it did with that freedom is one of the most instructive things in the entire history: **it decided everything should be a web service.**

Watch the architecture metastasize, commit by commit, over a single fortnight:

```
Websockets 1.0 implemented (May) -> Hardware Migration Work - Sockets 1.0 -> New WebSocket Centralized Management -> Phase 1 - Websockets 2.0 -> WS 2.1 -> Websockets 3.0 - fully individual RPI4bs -> MASSIVE Sockets Improvement for PArts -> WORKING Servos and Servo Services -> major services revision -> Services for ALL!
```

By `Services for ALL!` (Sept 6) the model had wrapped *every part of the animatronic* in its own service with its own port — `Fix servo and webcam service startup - always start these critical services for all characters`. Body parts — servos, the jaw, the eyes — each got a microservice. The repository was carrying **over 110 service/socket files**. This is exactly the architecture your CLAUDE.md now bans in capital letters: *"DO NOT introduce WebSockets, GraphQL, or new transport layers."* That rule is a scar from this exact night.

And it *was* a night. The commit timestamps tell the story: `Services for ALL!` lands Saturday Sept 6 at **19:51**, then the session grinds on — `Full Build` (21:29), `almost there...` (22:05), `Fix servo and webcam service startup` (22:59), `Fix critical syntax error` (23:21) — straight through into Sunday morning: `Big changes across` **(01:24)** and a single exhausted `good` **(01:33)**. An overnight AI session, too much autonomy, too young a model. Two days later the mood had curdled: `fucking sockers` (Sept 8 19:43), then simply `mess` (21:09).

The reset. The hangover came on Sept 13–14 and it was brutal. You started over: `first commit - MonsterBox4.0` (Sept 13), then `MONSTERBOX 4.0 CONVERSION - Single Node No Services` (Sept 14) — a single commit touching **770 files and deleting 13,101 lines.** The entire per-part microservice empire was torn

out and replaced with a single-node design. The simplification *worked* — within hours the log reads `WORKING LA - Servo Time`, `Fully Functional Servos!!!!!` — but it came at a real cost: **the reset lost features.** Rebuilding from a drastically simplified base meant capabilities that had been tangled up in the service layer didn't survive the conversion. (There's even a brief relapse a week later — `Final AI stretch - move BACK TO WEBSOCKETS`, Sept 21 — before the single-node design finally held.)

This era also shows AI's **churn** in microcosm: OpenAI gets fully integrated (`Fully Functional OpenAI WORKING`, Aug 23) and then deliberately ripped back out days later (`More cleanup - remove OpenAI and TOPMedia once and for all`, Aug 29).

The lesson is the thesis of this whole document, learned the hard way: **the model was powerful enough to build an entire distributed architecture overnight, and had no judgment about whether it should.** The capability was real; the restraint had to come from outside the model. Nothing yet stopped it from over-engineering, or from quietly breaking character independence — and there was no gate to catch either before it shipped. That second bill comes due in six weeks, on Halloween.

Act VI — The Halloween Reckoning (Oct–Nov 2025)

October 2025 starts strong and professional. A `.github/copilot-instructions.md` lands (Oct 22) and the workflow matures — `feat: integrate Copilot-driven testing with MCP tools`, `MCP Chrome Testing and Goblin finalization`. Conventional-commit prefixes (`feat:` / `fix:` / `chore:` / `docs:`) become the house style, and real version tags arrive in a steady cadence: `v5.3.0` (Oct 8), `v5.4.0` (Oct 23), `v5.5.0` (Oct 27). The **Goblin** distributed-media subsystem is designed here (`GOBLIN: comprehensive project design document`, Sept 28). It looks like the project has grown up.

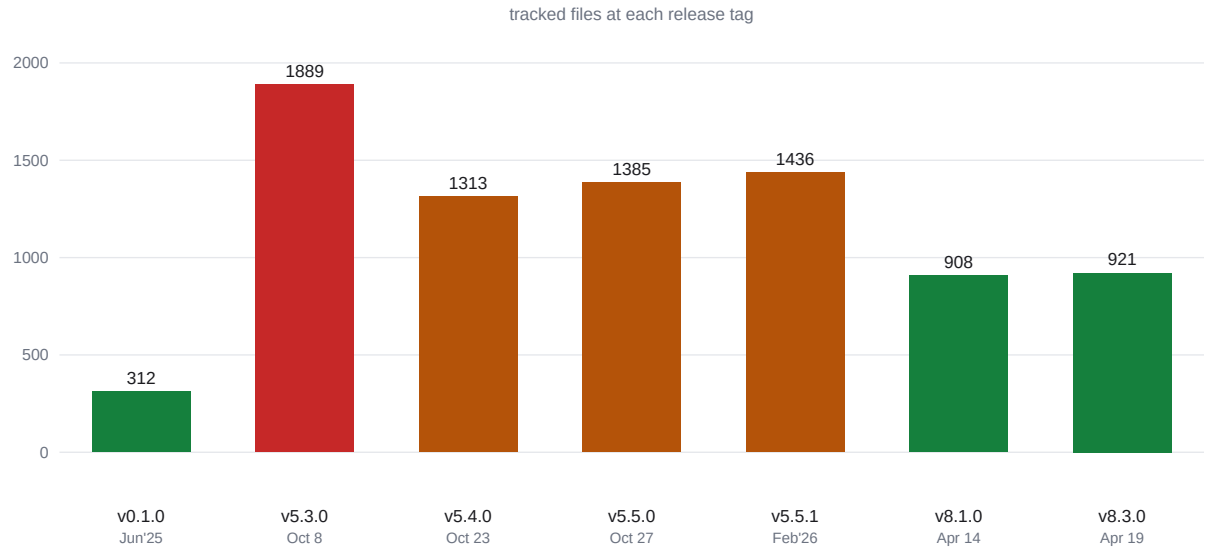
But it had also grown *bloated.* At `v5.3.0` the repository hit **1,889 tracked files** (717 JS, 506 test files) — see the spike in the file-count chart. That's what unconstrained AI generation produces: a flood of scripts, helpers, and tests, plus vendored `node_modules`. The very next tag, `v5.4.0` two weeks later, drops to **1,313** — a frantic pre-Halloween cleanup.

And then, on **Halloween night**, it falls over:

> `Fucked Halloween - nothing works Fucked Halloween EMERGENCY FIX: Restore hardware control for Halloween - remove test mode guards fix: filter global parts.json by characterId to avoid cross-character part conflicts EMERGENCY FIX: Restore all critical animatronic systems after Halloween failure`

This is the climax of the whole history, and the post-mortem is written right there in the fixes. Look at *what actually broke*: **test-mode guards** that blocked real hardware, and a **global** `parts.json` **leaking one character's parts into another.** Those are precisely the two failure modes the entire 2026 governance effort would be built to prevent: the gap between test and reality, and the collapse of character independence. The night of `nothing works` is the night the next year's roadmap got written in blood. The wound lingered, too — months later, in January: `Basic Update from Halloween post-nightmare`.

Act VII — The Governed Collaboration (2026)



Bloat, then discipline — tracked files per release

Everything in 2026 is a direct answer to Halloween. First, the swarm gets purged: on **Jan 6, 2026**, `.cursor/`, `.windsurf/`, `.roo/`, and `.augment/` are all deleted in one sweep — `Remove Augment and Cursor AI tool directories (no longer used)`. The era of *many* loosely-governed agents ends; the era of *one* governed collaborator begins.

The commit style reaches its final, mature form — a strict, versioned, scoped changelog grammar:

```
v6.8.0: [scene-concurrency] replace pair-based grouping with fire-and-forget concurrent model v7.0.0: [calibration] fix PIR sensor repeated detection and stuck indicator v8.1.4: [hotfix] untrack calibration_profiles.json — fixes per-node ID collision
```

Average message length peaks at **70 chars**, its highest ever. The software consolidates: the **Animation Studio** unifies the old Scenes and Poses pages (`v6.0.0: [animation-studio] Unified Animation Studio`, Feb 15), and the **Character Independence** project finally ships as a named release (`v6.0.0: [char-independence] Phase 1 complete`).

But the real transformation isn't the messages or the features — it's that **you stopped writing code and started writing the rules the AI writes code under.** - **Feb 14, 2026:** the first `CLAUDE.md` — `Add Claude Code integration with auto-restoration`. - **Feb 27:** `Smart MonsterBox Dev: shared memory, custom skills, CLAUDE.md updates`. - **Mar 22:** a versioned `.mcp.json`. - Then an entire **stabilization framework** lands as named pillars:

```
v8.1.8: [stabilization] Pillar 1 - schemas + validator v8.1.9: [stabilization]
Pillar 2 - canonical character resolver v8.2.0: [stabilization] Pillar 3 - character
pact suite v8.2.1: [stabilization] Pillar 4 - pre-deploy gate v8.2.2:
[stabilization] Pillar 5 - character-independence auditor v8.2.3: [stabilization]
Phase 6 - Claude Code primitives (subagents + skills)
```

Read those pillars against the Halloween post-mortem and the symmetry is perfect. The **character-independence auditor** exists because a global `parts.json` killed the show. The **pre-deploy gate** and **pact suite** exist because an "automated bug fix" with no guardrail had nothing to stop it. The **canonical resolver** exists because character context used to be read from a dozen hardcoded places. You took every 2024–2025 wound and turned it into an automated check the AI now *cannot* commit past.

And the codebase got **smaller**: from 1,889 files at the v5.3 peak down to **908** at `v8.1.0`. Growth stopped being the goal; *governed simplicity* became it. By 2026 there are **zero** frustration markers in the commit log — not because the work got easier, but because the swearing moved out of the commits and into the design of the guardrails.

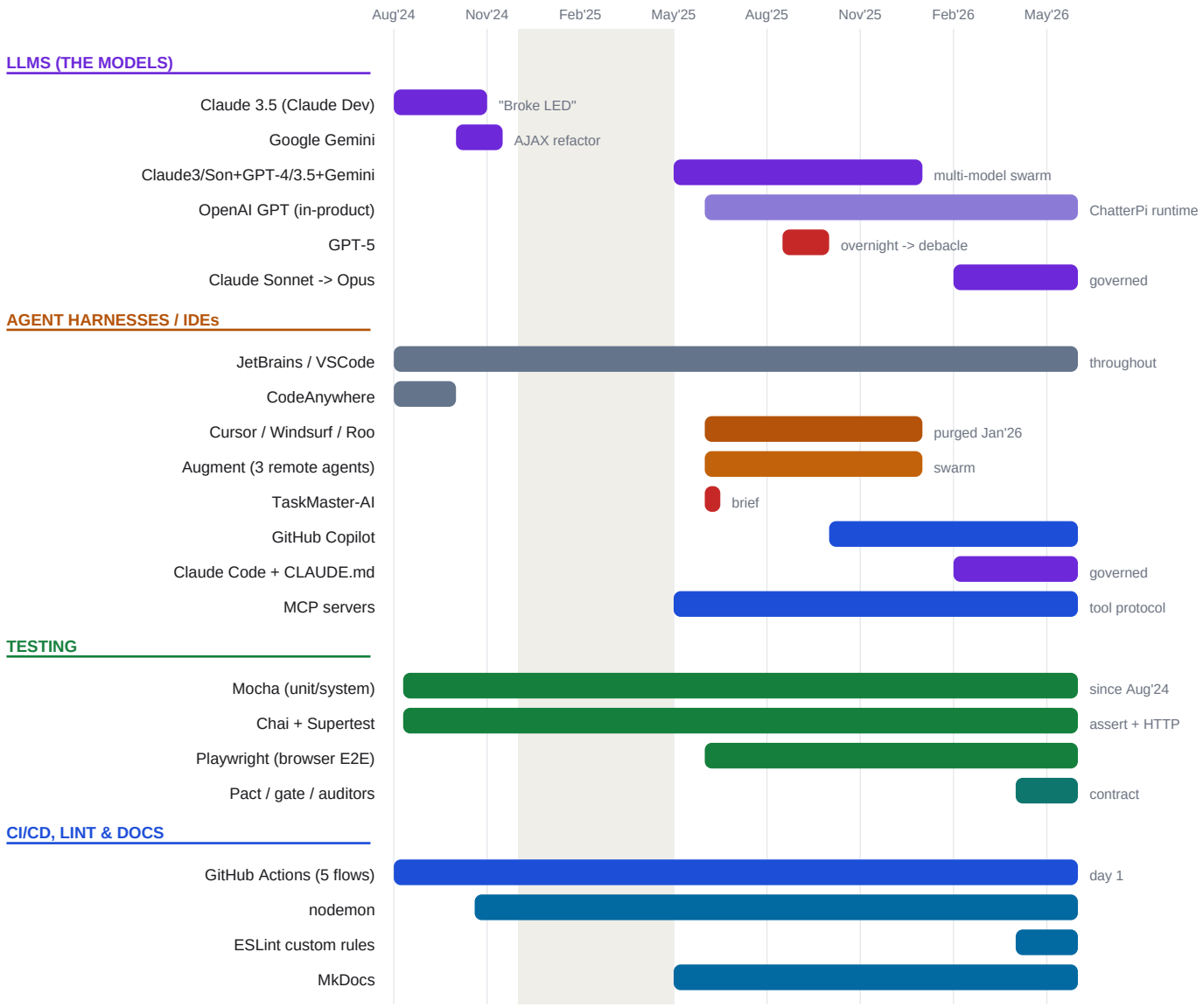
The Models Behind the Curtain

The story above is about *roles*; this is about the *named models* that played them. Each is datable from the commit log, the config files, or both:

When	Model(s) in play	Tool / harness	Evidence
Aug 2024	**Claude 3.5 Sonnet**	"Claude Dev" VSCode extension (early Cline)	`First ClaudeDev Update` (Aug 24); `End of Claude: Broke LED and Scenes` (Aug 18)
Oct 2024	**Google Gemini** + Claude	Cline	`Gemini AI Model` (Oct 12); `Geminis commits from AJAX removal` (Oct 14); `Cline switch voices` (Oct 27)
May–Jun 2025	**A multi-model swarm — Claude 3 / Claude Sonnet, GPT-4, GPT-3.5, Gemini**	Cursor, Windsurf, Roo, Augment + task-master-ai over MCP	model IDs `claude-3`, `claude-sonnet`, `gpt-4`, `gpt-3.5`, `gemini` all first appear in config files June 5–6, 2025; `Three Independent Augment Remote Agents`
Jun 2025 (runtime)	**OpenAI GPT** *(in the product, not just the IDE)*	ChatterPi	`ChatterPi Interactive Conversation System` (Jun 8) — the animatronics' own voice
Aug–Sep 2025	**GPT-5** *(newly released)*	semi-autonomous overnight sessions	`Major automated bug fix GPT5` (Sep 1) -> the microservices debacle
Oct 2025	**GitHub Copilot's models**	Copilot + MCP testing	`feat: integrate Copilot-driven testing with MCP tools` (Oct 22)
2026	**Claude Sonnet, then Claude Opus**	Claude Code, under `CLAUDE.md`	`claude-sonnet` in config; `claude-opus` first appears Feb 18 2026; the governed era

Two patterns jump out. First, the project was **never loyal to one vendor** — it rode whichever model was strongest for the job at the time, and at its peak (mid 2025) it was orchestrating Claude, GPT, and Gemini *simultaneously* through agent frameworks. Second, the **failures track the youngest models given the most freedom**: the Gemini-era AJAX churn, and above all GPT-5's overnight microservice sprawl. The models that caused the most damage weren't the weakest — they were the *newest and least-governed*. Capability arrived before judgment, every time, and the human's job became supplying the judgment the model lacked.

The Real Story: How GPT, Claude, and Gemini Changed in How They Were Used



The tools that built it, grouped by kind

The table above says *which* models. The more interesting story is *how the way of using them changed* — because the models got more capable and the interaction pattern climbed the abstraction ladder at the same time. MonsterBox rode all three major families. None of them was used the same way twice.

Claude, the first hands on the keyboard (2024). The project's first AI was Claude 3.5, reached through the "Claude Dev" VS Code extension (the tool that would become Cline). The mode of use was the most primitive one possible: *complete this block.* You described a function, Claude wrote it, you pasted it in and wired it up by hand. It had no view of the wider system, which is why it cheerfully broke the LEDs and the scene player —

`End of Claude: Broke LED and Scenes`. Claude here was a very fast intern who could only see the file in front of it.

Gemini, the specialist tried mid-stream (Oct 2024). For a stretch in October 2024 the work shifted to Google's Gemini (`Gemini AI Model`, `Geminis commits from AJAX removal`) — notably for a specific, mechanical refactor: tearing AJAX out of the front end. This is an early glimpse of a pattern that would define the next year: *picking a model for a job* rather than marrying one. Gemini was the tool you reached for when you wanted a large, repetitive transformation done quickly.

The multi-model swarm, where models became interchangeable labor (mid-2025). This is the inflection point. By June 2025 the config files wired up Claude 3 / Sonnet, GPT-4, GPT-3.5, *and* Gemini simultaneously, behind a fleet of agent harnesses — Cursor, Windsurf, Roo, and Augment — all driven through MCP against a numbered task backlog in TaskMaster. The mode of use jumped two rungs: from "complete this block" past "write this function" all the way to *"own this task."* The model stopped being something you talked to and became a *worker you dispatched* — three Augment agents running in parallel, each grinding through tickets and reporting `Complete Task 17 … 100% DONE`. Crucially, the individual model became almost a commodity: the leverage now lived in the *harness* (the agent framework, the MCP tools, the task list), not in which logo answered.

GPT as a product ingredient, not just a dev tool (2025). In parallel, OpenAI's GPT crossed a line none of the others had: it went *inside the animatronics.* The ChatterPi conversation system used GPT at runtime so the characters could improvise speech to a live guest. For the first time a model wasn't building the product — it *was* a feature of the product. (This, too, churned: OpenAI was integrated, then ripped out, then the project standardized its speech stack — a reminder that "which model powers the feature" was as unsettled as "which model writes the code.")

GPT-5 as an autonomous agent, trusted too far (Sept 2025). When GPT-5 arrived it was handed the most autonomy yet — semi-unsupervised, overnight. The mode of use had quietly become *"let it run and check in the morning,"* and the model, with no instinct for restraint, used that freedom to wrap every body part in its own web service. The capability was staggering; the judgment was absent. The lesson wasn't "GPT-5 is bad" — it's that *the more autonomy you grant, the more judgment you have to supply from outside the model,* and that hadn't been built yet.

Copilot, the test-driven IDE partner (late 2025). After the GPT-5 hangover the pendulum swung back toward supervision: GitHub Copilot, wired to MCP testing tools, used in a tighter loop where changes were validated as they were made (`integrate Copilot-driven testing with MCP tools`). The mode of use was narrower and safer by design — the model proposes, the tests dispose.

Claude again, but governed (2026). The project came full circle to Claude — now Sonnet and Opus through Claude Code — but the *relationship* was unrecognizable from 2024. The 2024 Claude was an intern you watched constantly. The 2026 Claude is a senior agent you trust *because it operates inside a constitution you wrote:* a `CLAUDE.md` of hard rules, schemas it must satisfy, a resolver it must use, a pre-deploy gate it cannot commit past, and its own subagents and skills. Same vendor, same family — completely different mode of use. The human went from *supervising every line* to *designing the rules of engagement.*

So the journey of GPT / Claude / Gemini isn't really a story about which model "won." It's a story about the **interaction pattern maturing through five stages** — *complete the line -> write the function -> own the task -> run unsupervised -> operate under governance* — with the models powerful enough to reach the next rung long before anyone had built the guardrails that rung required. The vendor mattered less than the harness around it, and the harness mattered less than whether a human had yet encoded the judgment the model still lacked.

The Arc: how AI's role actually changed

Strip away the dates and one clean progression remains — the entire industry's journey, compressed into twenty months:

> **1. Autocomplete you babysit (2024).** AI generates a block; you integrate it by hand and clean up when it breaks the LEDs. -> `End of Claude: Broke LED and Scenes.` 2. **A narrator of its own work (mid-2025).** AI writes whole changes and describes them clearly. Commit messages triple in length overnight. 3. **A managed workforce (mid-2025).** Three agents run in parallel against a task backlog over MCP. You become a manager of AI labor. -> `Three Independent Remote Agents.` 4. **An automated fixer (late 2025).** AI lands sweeping fixes alone — fast, powerful, untrustworthy without judgment. -> `Major automated bug fix GPT5.` 5. **A governed collaborator (2026).** AI works inside schemas, a resolver, a contract suite, and a pre-deploy gate *you* designed — plus its own subagents and skills. Fast *and* safe, because the safety lives in the system, not in your vigilance.

The deepest lesson MonsterBox teaches is that **the bottleneck moved.** In 2024 the constraint was AI's raw capability — it couldn't build much without breaking things. By 2026 capability is abundant; the constraint is *governance.* What made AI genuinely productive here wasn't a better model. It was a human who got tired of `Fucked Halloween`, sat down, and built the guardrails — the schemas, the resolver, the gate, the auditor, the `CLAUDE.md` — that let a powerful, careless tool finally be trusted with a real machine that has to work on the one night a year it matters.

You started by typing `fff` at midnight. You ended by writing the constitution an AI has to obey before it's allowed to commit. The animatronics were always the point — but what you really built was the *discipline* that makes building with AI sustainable.

Lessons for Anyone Building With AI in 2026

MonsterBox is one project by one person, but the scars generalize. Here is what two years and 2,020 commits taught — each lesson paid for in a `Fucked Halloween`:

> **1. Capability arrives before judgment — every time.** The newest, most powerful models did the most spectacular damage (GPT-5 building a microservice for every body part overnight), not because they were dumb but because they had no instinct for restraint. *Assume the model can do far more than it should, and that knowing the difference is your job, not its.*
>
> **2. Autonomy without guardrails is a loan, not a gift.** The overnight "let it run" session felt like a windfall and cost a 770-file, 13,000-line teardown and lost features. Give an agent more rope only as fast as you

build the checks that catch it. Unsupervised AI is fastest at writing code you'll later have to delete.

3. Put the safety in the system, not in your vigilance. Nothing improved reliability until the guardrails became *automated and unskippable* — schemas, a canonical resolver, a contract test suite, a pre-deploy gate. Reviewing AI output by eye does not scale; a gate the AI literally cannot commit past does. The reliability win was a *governance* win, not a model upgrade.

4. Write the constitution down where the AI can read it. A `CLAUDE.md` that says "DO NOT introduce new transport layers" turns a hard-won scar into a standing rule the assistant honors on every future task. Encode your architectural decisions as machine-readable constraints, not tribal knowledge.

5. Beware the one load-bearing default. So much was hardcoded to Orlok that "character independence" became a multi-month crusade. AI will happily cement whatever it sees first into an assumption everywhere. Design for the general case early, or pay to retrofit it later.

6. Smaller is a feature. AI generation inflated the repo to 1,889 files; the healthiest era cut it nearly in half. More code is not more progress — under AI it's often the opposite. Deletion and consolidation are real work, and the most mature commits in this history are the ones that *removed* things.

7. Commit messages are a cheap, honest instrument. The single clearest signal of how the work was actually going — tooling, discipline, even mood — was hiding in the commit log the whole time. Write them like they'll be read later, because the story of your project is being recorded in them whether you mean it to be or not.

8. The human's job changed, but didn't shrink. Across two years the human stopped writing most of the code — and became *more* essential, not less. The memory, the judgment, the questions, the decision of *what's worth doing and what must never happen* stayed stubbornly human. AI got better at the *how*; the *whether* and the *why* are still yours.

The throughline: **AI in 2026 is powerful enough to build almost anything and wise enough to build almost nothing.** The leverage comes from a human supplying the judgment — and, increasingly, from encoding that judgment into systems the AI must obey. That's not a limitation to wait out. It's the job.

Sources: full `git log` of 2,020 commits (2024-08-15 -> 2026-04-19). Quantitative signals: monthly commit counts; per-month average commit-subject length; lexical sentiment tally (frustration vs. triumph word families); config-directory add/remove lifecycles for every dev tool (`.idea`, `.vscode`, `.cursor`, `.windsurf`, `.roo`, `.augment`, `.taskmaster`, `.claude`, `.mcp.json`, `CLAUDE.md`, `copilot-instructions.md`); 227 AI co-author/generated-by trailers; tracked-file counts and language mix at each release tag; and first-appearance dates for each major feature (TTS, head tracking, ChatterPi conversation, STT, jaw animation, Goblin, Animation Studio, character independence). Companion visual dashboard: `docs/THE-MONSTERBOX-STORY.html`.

Appendix — How This Document Was Written (Together)

One last twist worth naming: **this story about building with AI was itself built with AI** — and it couldn't have been written any other way.

The raw material was 2,020 commits over twenty months. No human reads 2,020 commit messages, tallies the swear words by month, cross-references config directories against their deletion dates, computes the average subject length per month, diffs the file count at every release tag, and traces one character's name through hundreds of JSON revisions. It's not hard — the volume just puts it past human patience. The data was always there, in the open, in the repository. It was simply **unreadable at that scale by the person who created it.**

So the division of labor went like this:

- **The human brought the memory and the meaning.** Aaron knew there *was* a Baphomet, suspected there was a forking moment, remembered an overnight session that went wrong, recalled "too much control to an AI that was too young." Those are the threads — the things worth looking for. None of them were obvious from the data alone; they came from having *lived* it. - **The AI brought the reading and the recall.** Claude unshallowed the git history, ran dozens of queries across the full 2,020 commits, surfaced the exact timestamps of the overnight microservice session, found that "Scary Pete" preceded Baphomet by a day, measured the 13->70 character climb, and pulled the verbatim quotes — `embracesuck`, `Services for ALL!`, `Fucked Halloween`, `good` at 01:33 — that make the story *feel* true because it *is* true. - **Then it became a conversation.** The human read what the AI found, recognized some of it, corrected the rest ("it wasn't always MonsterBox," "I lost features in the reset"), and pointed at the next thread. The AI went back to the data and came back with evidence. Five rounds of that built this document.

That loop is the whole point. In 2024, AI broke the LEDs because it couldn't see the whole system. In 2026, AI can read the *entire history* of that system in minutes and tell you true things about it you'd never have assembled yourself — but only because a human steered it toward the parts that mattered. The same partnership that now ships animatronic code under a pre-deploy gate wrote its own origin story: **the human supplies the memory, the judgment, and the question; the AI supplies the tireless reading; and the truth lives in the commits — invisible to one, inaccessible to the other — until the two work together.**

That's the artifact this document really is: a human story, about a human's project, that only a human and an AI could have told together.